Generation Routing, Hedging, Failover, and Validation

This document describes the Supabase generation Edge Functions as currently implemented. It covers the five item-count generators and the separate pdf-generation handout route. Configuration can change; verify the referenced source when using this document to interpret later telemetry.

Functions and provider order

The item-count functions are:

? generate-quiz
? generate-flashcards
? generate-matching
? generate-mixed-quiz
? generate-true-or-false

They try Groq models before Gemini models. PDF handout generation is separate: it tries Gemini models first, followed by Groq.

Provider and model selection rationale

The quota comparison provided for this project lists the following free-tier limits. Provider quotas are account- and project-dependent and may change; use the current provider console as the operational source of truth.

Provider/model	RPM	RPD	TPM	TPD
Groq — GPT-OSS 120B	30	1,000	8,000	200,000
Groq — GPT-OSS 20B	30	1,000	8,000	200,000
Groq — Qwen 3.8 27B	30	1,000	8,000	200,000
Google — Gemini 3.5 Flash-Lite	15	500	250,000	Not published
Google — Gemini 3.1 Flash-Lite	15	500	250,000	Not published

TPM is the provider's tokens-per-minute quota, not a guarantee of response speed or availability. Although the listed Gemini TPM is much higher, the project's testing and operational experience found Gemini availability less predictable under high demand. For ordinary quiz and learning-item generation, Groq is therefore tried first: lower observed latency and the listed higher request-per-minute limit were more useful than Gemini's larger TPM allowance. Gemini remains a fallback.

The provided latency summary covers 291 attempts across the five listed models. It does not show the per-model attempt counts or full test conditions, so treat these numbers as observed results from that test set, not a controlled or permanent ranking:

Model	Average latency (ms)	P50 latency (ms)	P90 latency (ms)
Gemini 3.1 Flash-Lite	3,770.70	3,567.5	6,124.3
Gemini 3.5 Flash-Lite	4,976.45	4,382	8,093

GPT-OSS 120B



2,598.76

2,298

5,635.3

Model	Average latency (ms)	P50 latency (ms)	P90 latency (ms)
GPT-OSS 20B	2,348.76	2,590	3,441
Qwen 3.8 27B	4,375.8	5,709	8,165

In this sample, both GPT-OSS models had lower average latency than either Gemini model. Qwen did not: its average and P90 were higher than Gemini 3.1 Flash-Lite. Thus the results support preferring Groq's GPT-OSS models for latency-sensitive item generation, but do not show that every Groq model is faster than Gemini.

These model latency measurements are not the complete time an end user experiences. Network quality between the user's device and the application, request upload/download time, server processing, and other connection conditions also affect end-to-end latency. Users on slower or unstable connections may see longer waits even when the provider-side model latency is low.

PDF handout generation is Gemini-first for a different reason. Handouts request substantially more prose than a typical quiz batch, so Gemini's listed 250,000 TPM offers much more input/output-token quota headroom than Groq's 8,000 TPM. This makes Gemini's quota advantage useful for longer handout generation even though Gemini was not the latency leader in the supplied test summary. It does not remove the effects of request-per-minute limits or provider availability.

Groq API key priority

Each model route uses its function's configured key order. All five Groq keys are available to the item-count functions:

Function	Key order, highest priority first
Quiz	GROQ_API_KEY_5, _2, _1, _3, _4
Flashcards	GROQ_API_KEY_1, _2, _3, _4, _5
True/false	GROQ_API_KEY_1, _2, _3, _4, _5
Matching	GROQ_API_KEY_4, _3, _1, _2, _5
Mixed quiz	GROQ_API_KEY_4, _3, _1, _2, _5

Only configured keys can be selected. Among configured, healthy, untried keys, the planner uses this order; it does not rank keys by current in-flight load. Gemini keys are considered only after Groq model routes.

Candidate planning and failover

For each request, the function creates a planner with an ordered list of model routes. For each model, it tracks which API keys have already been tried:

1. Pick the next model route in priority order.
2. Select the first configured key that is not cooling down and has not already

been tried for that model.

1. If an attempt fails, continue to another eligible key or model.
2. If the provider returns HTTP 404, mark that model as unavailable for the

rest of this request and skip its remaining keys.

1. Stop when a valid result wins, the attempt/request limits are reached, the

client disconnects, or no eligible candidate remains.

The item-count functions permit up to five keys per model and up to 25 launched attempts per request. With

three Groq models and two Gemini models, each with five possible keys, that is the maximum possible model/key combinations, not a promise they will all run. The 55-second request budget, cooldowns, cancellation, and availability determine how many actually launch.

Error cooldowns

Failures can put a key/model pair or key into cooldown so the planner avoids repeating a known failing route immediately. Current base durations include:

Failure	Base cooldown
Rate limit (HTTP 429)	20 seconds, or provider retry delay
Overload/network	5–8 seconds
Bad request	60 seconds
Model not found (HTTP 404)	5 minutes effective for that model/key
Key authentication failure (HTTP 401/403)	10 minutes for that key

Rate-limit and overload responses can supply an explicit provider retry delay; that delay is capped at 5 minutes. Otherwise rate-limit, overload, and network cooldowns increase exponentially on repeated failures up to a 5-minute cap. Bad-request cooldown is fixed at 60 seconds. Although the not-found base is configured as 10 minutes, the shared 5-minute cap makes its effective cooldown 5 minutes. Authentication errors disable the key for 10 minutes. Unreadable provider JSON is treated as a network cooldown; other structurally invalid generated output is recorded as invalid output and does not itself put the route into cooldown.

Hedging

Parallel hedging is a latency strategy: rather than waiting for one slow provider call to time out, the function can launch another candidate while the first is still running. It differs from sequential failover, which starts the next candidate only after the previous attempt fails.

The flow is:

1. Launch the highest-priority available model/key attempt.
2. While it is pending, wait for its hedge delay. If it returns a valid result

first, return that result without launching a hedge.

1. If the delay expires and the request has capacity, launch the next candidate

from the same ordered planner. Both provider calls now run concurrently. The next candidate may be another key for the same model before the planner advances to a different model.

1. Accept the first response that passes output validation and cancel any

slower pending attempt. If an attempt fails and no other attempt is pending, launch the next candidate immediately instead of waiting for a hedge timer.

The concurrency controls are:

? No more than two attempts are in flight for one request.

? No more than two timer-triggered hedges are launched per request. Sequential

failover attempts after completed failures are controlled separately by the overall attempt limit.

? A timer-triggered hedge is skipped when the Edge Function isolate already

has 20 or more provider attempts in flight.

? The timer is measured from the oldest pending attempt, and each new attempt

must fit within the request's remaining time budget.

For example, with a 7-second hedge delay, if attempt A is still pending at 7 seconds, attempt B starts while A continues. If B returns valid output at 9 seconds, the function returns B and aborts A. If A returns valid output at 8 seconds, A wins and B is never launched. If both fail validation, the function continues through normal failover.

Running two calls can improve tail latency, but may consume additional provider quota and compute because both calls can generate output. Cancellation is best effort; it cannot guarantee the provider did not already process or bill the aborted request.

For item-count functions, the current timing settings are:

Requested items	Hedge delay	Per-attempt timeout
1–10	7 seconds	15 seconds
11–20	10 seconds	22 seconds
21–30	14 seconds	30 seconds

The overall request budget is 55 seconds. A new attempt is not started when less than 4 seconds remain. PDF generation has separate timing and routing settings.

PDF handout generation uses a 50-second request budget, 24-second per-attempt timeout, 4-second minimum remaining budget to start an attempt, 8-second hedge delay, two simultaneous attempts, and at most two timer-triggered hedges. It is limited to eight total attempts and two keys per model. The PDF route order is gemini-3.5-flash-lite, gemini-3.1-flash-lite, then Groq openai/gpt-oss-120b, openai/gpt-oss-20b, and qwen/qwen3.8-27b. It loads Gemini keys 1–5 and Groq keys 1–3. If a provider rejects an optional speed setting with HTTP 400, that setting is disabled for that model and the request can retry without it. A per-isolate topic cache can also return a recently generated handout without making a provider attempt.

Output validation

Validation checks that the response can safely meet the function's expected output structure. It does not prove the generated content is factually correct, unambiguous, pedagogically sound, or fully supported by the source.

Function	Checks before accepting output
Quiz	Root contains only questions; exact requested count; each question contains exactly question, options, answerIndex, and explanation; non-empty question/explanation/options; exactly four options; integer answer index from 0 to 3; no duplicate options after trimming and case normalization.
Flashcards	Requires a cards array and at least the requested number of valid cards. Each accepted card has non-empty question, answer, explanation, and hint. Invalid cards are filtered; the first requested number of valid cards are returned.
Matching	Requires a pairs array and at least the requested number of valid pairs. Each pair has non-empty left, right, and explanation. Among the returned first requested pairs, left values and right values must each be unique after trimming and case normalization.

Function	Checks before accepting output
True/false	Requires at least the requested number of valid questions. Each has a non-empty question and explanation, exactly two non-empty options in True, False order (case-insensitive), and an integer answer index of 0 or 1. The first requested valid questions are returned.
Mixed quiz	Requires enough valid questions to meet the requested total and the required count of both types. Multiple-choice questions have four distinct options; true/false questions may use either order and are normalized to True, False with the answer index adjusted. The validator takes the required count by type and shuffles their final order.
PDF handout	Uses a separate, lenient normalizer. It trims/clamps text and list sizes, filters malformed subsections/sections/terms, deduplicates terms, and requires a title, summary, at least three sections, and at least four terms.

Mixed-quiz composition calculation

For a requested count n, the number of true/false questions is:

```
trueFalseCount = min(n - 1, max(1, round(n × 0.30)))
multipleChoiceCount = n - trueFalseCount
```

For example:

Total questions	True/false	Multiple-choice
1	0	1
10	3	7
20	6	14
30	9	21

JavaScript Math.round is used. A valid true/false pair is normalized without changing its meaning: if the model returns ["False", "True"], the validator switches the options to ["True", "False"] and maps the answer index to the new position of the same correct answer.

Definitions

? Attempt latency (latency_ms): elapsed time for one model/key attempt,

including provider response handling and output validation. It is not the entire user-visible request duration. The response handler awaits telemetry persistence before returning, so telemetry database time can add to perceived end-to-end latency.

? Winner: the attempt whose validated output is returned to the client.

? Parallel hedge: an additional model/key attempt launched while an earlier

attempt is still running, to reduce waiting on a slow provider response.

? Key failover: true when the attempt uses a configured key other than the

first configured key for that provider in that function.

? Validation failure: a provider response that cannot be accepted because

it is malformed or does not meet the generator's required output structure; the planner can continue to another candidate.

Operational notes

? Configure the API-key environment variables listed in each function's source.

Unconfigured keys are skipped.

? The item-count functions use Gemini only after their Groq model routes; PDF

generation is Gemini-first.

? The service validates structure before returning generated quiz/learning

content, but educational correctness still requires human review or a separate fact-checking process.

? Re-deploy the changed Edge Functions and re-run

supabase/quiz_generation_telemetry.sql when applying telemetry-view changes to Supabase.

Implementation references

? Quiz routing, planning, and validation
? Flashcard routing and validation
? Matching routing and validation
? Mixed-quiz routing and validation
? True/false routing and validation
? PDF handout routing and normalization
? Telemetry schema and views